

Multi-agent разработка

Как организовать несколько ИИ-агентов
вокруг задачи: без дублирования
работы, с понятными артефактами и
контролем качества.



Зачем вообще несколько агентов?

Multi-agent подход нужен не для того, чтобы параллельно "писать больше кода". Его ценность - развести мышление, проверку и реализацию по разным ролям.

Один агент часто смешивает четыре режима работы

- интерпретирует задачу и скрытые требования;
- выбирает архитектурный путь;
- вносит изменения в код;
- проверяет собственный diff и может не заметить ошибку.

Что даёт разделение ролей

● Более точная декомпозиция

До начала изменения кода видно зависимости, риски, порядок работ и вопросы к контрактам.

● Меньше ложной уверенности

Ревьюер и QA смотрят на результат отдельно, а не продолжают исходную гипотезу автора.

● Предсказуемый формат

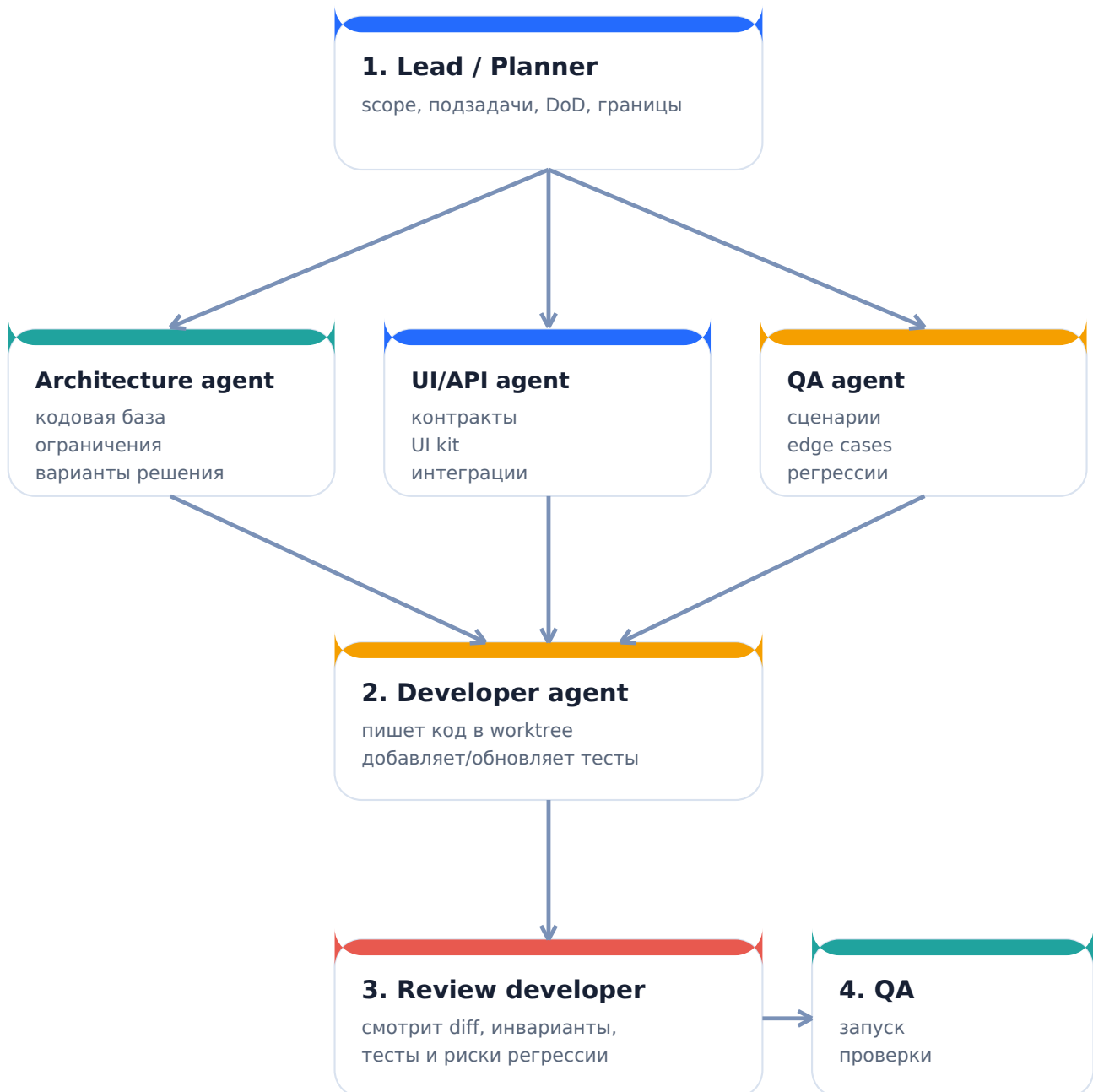
Каждая роль оставляет конкретный артефакт: план, решение, diff-review или тест-план.

Главный принцип

Роли должны быть независимыми по способу проверки, но не конкурировать за один и тот же участок реализации.

Целевой поток работы

Lead формирует задачу и критерии, исследовательские агенты снижают неопределённость, разработчик реализует изменения в отдельном worktree, затем результат проходит независимое ревью и QA.



Важное ограничение

Не запускайте всех агентов "на всё". Параллельность оправдана только в исследовании независимых областей; реализация должна иметь одного владельца.

Роли и ожидаемые артефакты

Агент полезен, когда его результат можно быстро прочитать, проверить и применить. Длинное рассуждение без решения - не артефакт.

Роль	Зона ответственности	Артефакт на выходе
Lead / Planner	Карта задачи и принятие решений	Цель, scope / out-of-scope, подзадачи, DoD, владельцы
Architecture agent	Проверка технического пути	Затронутые модули, альтернативы, риски, последовательность
UI/API agent	Контракты и границы интеграций	API / типы / состояния UI, требования UI kit, миграции
QA agent	Стратегия проверки	Основные сценарии, edge cases, регрессии, тестовые данные
Developer agent	Одна согласованная реализация	Diff, тесты, локальные проверки, описание изменений
Review developer	Независимая проверка изменения	Замечания по корректности, стилю, рискам и покрытию

Нормальный формат ответа от исследовательского агента

1) краткий вывод; 2) факты из репозитория с путями; 3) решение или 2-3 варианта; 4) риски; 5) что передать разработчику.

Когда это уместно, а когда избыточно

Полная цепочка из шести ролей не должна быть процессом по умолчанию. Масштаб выбирается по риску и неопределённости задачи, а не по размеру команды.

Уровень 1 - простая задача

Один **developer agent + self-check**

Локальный багфикс, мелкий UI, изолированный рефакторинг.

Обычно достаточно 1 цикла.

Уровень 2 - средняя задача

Lead + developer + reviewer

Новая интеграция, несколько модулей, изменения в shared-коде.

Практичный базовый режим.

Уровень 3 - высокая неопределённость

Lead + 2-3 research agents + developer + review/QA

Новый домен, критичный поток, неясные контракты, заметный риск регрессий.

Полная схема оправдана.

Антипаттерн: роль ради роли

Если архитектурный агент просто пересказывает уже известный код, QA пишет общие фразы, а review запускается без изменений - цепочка замедляет работу и создаёт иллюзию контроля.

Как организовать это в Cursor / CLI-agent

Необязательно строить сложную платформу. Сначала договоритесь о контексте, форматах результатов и изоляции реализации.

1

1. Единый входной документ

Lead создаёт task brief: цель, ссылки на код, ограничения, DoD и явные вопросы.

2

2. Короткие специализации

Для исследовательских агентов задайте фиксированный формат: вывод, файлы, риски, рекомендации.

3

3. Один owner у кода

Developer работает в отдельном worktree или ветке. Никто больше не вносит параллельные изменения в тот же scope.

4

4. Review по diff, не по истории

Ревьюер получает задачу, итоговый план и diff. Его цель - найти расхождения между замыслом и реализацией.

5

5. QA от критериев готовности

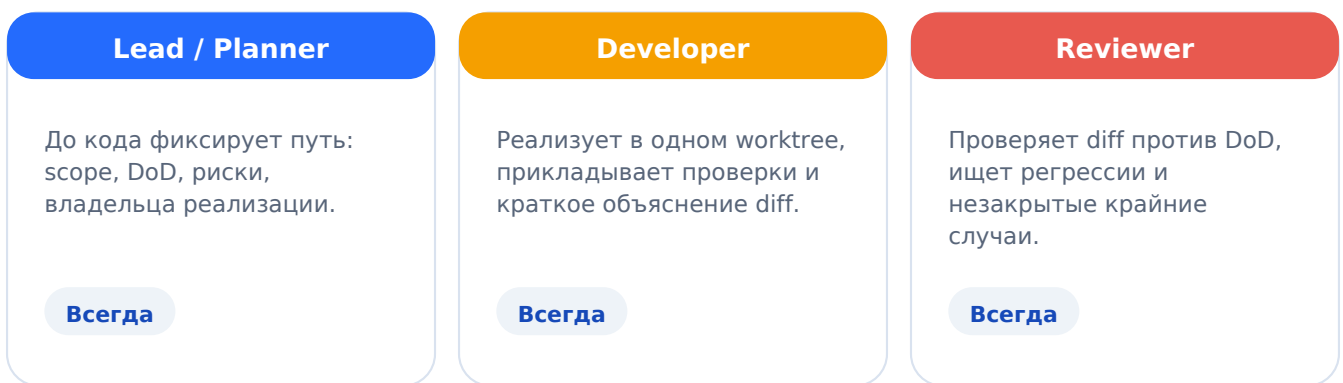
Проверка начинается с DoD и пользовательских сценариев, а не с абстрактного списка тестов.

Минимальный task brief

- Цель: что изменится для пользователя или системы.
- Scope: какие модули / экраны / API затрагиваются; что сознательно не меняем.
- Constraints: совместимость, UI kit, feature flag, ограничения релиза.
- Definition of Done: наблюдаемые критерии готовности и обязательные проверки.

Рекомендуемый стартовый вариант

Ваша схема хороша для сложных задач, но как постоянный режим она будет избыточной. Начните с трёх обязательных ролей и подключайте исследования / QA по триггерам.



Подключайте дополнительные роли по триггерам

